크레이트 구조 (Crate Structure)

개요

Scroll ZKEVM은 Rust의 workspace 구조를 사용하여 여러 크레이트로 모듈화되어 있습니다. 각 크레이트는 명확한 책임을 가지며, 의존성 관계를 통해 협력합니다.

Workspace 구조

```
# Cargo.toml

[workspace]

members = [

"zkevm-circuits", # 모든 회로 구현

"bus-mapping", # 트레이스 → 회로 입력 변환

"geth-utils", # Geth 트레이서 연동

"zktrie", # zkTrie 구현

"gadgets", # 재사용 가능한 회로 컴포넌트

"integration-tests", # 통합 테스트

"circuit-benchmarks", # 회로 벤치마크

"eth-types", # 이더리움 타입 정의

"external-tracer", # 외부 트레이서

"mock", # 테스트 유틸리티
```

```
"testool", # 공식 테스트 실행
"aggregator", # 증명 집계
"prover" # 증명 생성
]
```

크레이트별 상세 설명

1. zkevm-circuits

역할: 모든 영지식 증명 회로의 구현을 포함하는 핵심 크레이트

주요 모듈:

```rust
// zkevm-circuits/src/lib.rs
pub mod bytecode_circuit; // 바이트코드 검증
pub mod copy_circuit; // 복사 연산 검증
pub mod ecc_circuit; // 타원곡선 연산
pub mod evm_circuit; // EVM 실행 검증
pub mod exp_circuit; // 지수 연산
pub mod keccak_circuit; // Keccak 해시
pub mod mpt_circuit; // Merkle Patricia Trie
pub mod pi_circuit; // Public Input
pub mod poseidon_circuit; // Poseidon 해시
pub mod rlp_circuit_fsm; // RLP 인코딩
pub mod sig_circuit; // 서명 검증
```

```rust
pub mod modexp_circuit; // 모듈러 지수 연산
pub mod sha256_circuit; // SHA256 해시
pub mod state_circuit; // 상태 관리
pub mod super_circuit; // 통합 회로
pub mod table; // 공유 테이블
pub mod tx_circuit; // 트랜잭션 검증
pub mod util; // 유틸리티
pub mod witness; // Witness 생성
```

주요 의존성:

- `halo2_proofs` : 영지식 증명 백엔드
- `bus-mapping` : 트레이스 변환
- `eth-types` : 이더리움 타입
- `gadgets` : 재사용 가능한 회로 컴포넌트
- `halo2-base` , `halo2-ecc` : halo2-lib 라이브러리
- `mpt-circuits` : MPT 회로 (외부)
- `misc-precompiled-circuit` : 프리컴파일 회로 (외부)

특징:

- Scroll 특화 기능: SHA256, MODEXP, ECC 프리컴파일
- zkTrie 지원
- Poseidon 코드해시 지원
- 듀얼 바이트코드 테이블 지원

2. bus-mapping

역할: EVM 실행 트레이스를 회로 입력으로 변환

주요 구조체:

```rust
// bus-mapping/src/circuit_input_builder.rs
pub struct CircuitInputBuilder {
pub sdb: StateDB, // 상태 데이터베이스
pub code_db: CodeDB, // 코드 데이터베이스
pub block: Blocks, // 블록 데이터
pub block_ctx: BlockContext, // 블록 컨텍스트
}
```

주요 기능:

- Geth 실행 트레이스 파싱
- Opcode별 RW 연산 생성
- ExecStep 생성
- 상태 변경 추적

주요 의존성:

- `eth-types` : 이더리움 타입
- `gadgets` : 회로 컴포넌트
- `mpt-zktrie` : zkTrie 구현
- `revm-precompile` : 프리컴파일 구현

특징:

- Scroll L2 특화 기능
- 트레이스 재생성 기능 (`retrace-tx`)

- 메모리/스택/스토리지 추적 옵션

3. eth-types

역할: 이더리움 관련 타입 정의 및 유틸리티

주요 타입:

- `Block`, `Transaction`: 블록 및 트랜잭션
- `GethExecTrace`, `GethExecStep`: Geth 트레이스
- `Word`, `Address`, `Hash`: 기본 타입
- `OpcodeId`: Opcode 열거형
- `GasCost`: Gas 비용

주요 의존성:

- `ethers-core`: 이더리움 코어 타입
- `halo2curves`: 타원곡선 연산
- `poseidon-base`: Poseidon 해시
- `revm-precompile`, `revm-primitives`: EVM 구현

특징:

- Scroll L2 타입 확장
- 트레이스 힙 할당 추적 옵션

4. gadgets

역할: 재사용 가능한 회로 컴포넌트 (Gadget) 제공

주요 Gadget:

- `IsZeroGadget`: 값이 0인지 검증
- `IsEqualGadget`: 두 값이 같은지 검증

- `LtWordGadget` : 대소 비교
- `RangeCheckGadget` : 범위 검증
- `MulAddWordsGadget` : 곱셈-덧셈 연산

주요 의존성:

- `halo2_proofs` : Halo2 백엔드
- `eth-types` : 이더리움 타입
- `poseidon-base` : Poseidon 해시

특징:

- 회로 간 공통 로직 재사용
- 최적화된 제약 조건 구현

5. geth-utils

역할: Geth 트레이서와의 연동

주요 기능:

- Geth 빌드 및 연동
- 트레이스 수집
- L1/L2 Geth 지원

구조:

```
geth-utils/
├── l1geth/ # L1 Geth 서브모듈
├── l2geth/ # L2 Geth 서브모듈
└── src/ # Rust 바인딩
```

특징:

- Go 코드와 Rust 바인딩
- 커스텀 트레이서 구현

6. mock

역할: 테스트용 Mock 데이터 생성

주요 기능:

- Mock 블록 생성
- Mock 트랜잭션 생성
- Mock 실행 트레이스 생성

주요 의존성:

- `eth-types` : 이더리움 타입
- `ethers-core` : 이더리움 코어

특징:

- 단위 테스트 지원
- Scroll L2 Mock 데이터

7. testool

역할: 이더리움 공식 테스트 실행

주요 기능:

- 공식 테스트 벡터 실행
- 회로 검증
- 테스트 결과 리포팅

특징:

- Docker 기반 실행 환경
- 대규모 테스트 자동화

8. aggregator

역할: 여러 증명을 하나로 집계

주요 기능:

- 청킹 전략 구현
- 증명 집계
- 재귀적 증명 생성

주요 의존성:

- `snark-verifier` : 증명 검증
- `snark-verifier-sdk` : SDK

특징:

- 멀티 블록 청킹
- 최종 증명 생성

9. prover

역할: 개별 회로의 증명 생성

주요 기능:

- Setup 파라미터 관리
- 증명 생성
- 증명 검증

특징:

- Halo2 백엔드 사용

- 병렬 증명 생성

10. zktrie

역할: zkTrie (Sparse Binary Merkle Patricia Trie) 구현

주요 기능:

- zkTrie 상태 관리
- Poseidon 해시 기반 트리
- 증명 생성

특징:

- Scroll L2 전용
- 효율적인 상태 증명

11. external-tracer

역할: 외부 트레이서 연동

주요 기능:

- 외부 노드에서 트레이스 수집
- 트레이스 재생성

특징:

- 다양한 노드 지원
- 트레이스 검증

12. integration-tests

역할: 통합 테스트

주요 기능:

- 전체 시스템 테스트
- 회로 통합 검증
- 성능 테스트

특징:

- Docker Compose 기반
- 실제 트랜잭션 테스트

13. circuit-benchmarks

역할: 회로 성능 벤치마크

주요 기능:

- 회로별 성능 측정
- 리소스 사용량 분석
- 최적화 가이드

의존성 그래프

```
┌──────────────┐
│ zkevm- │
│ circuits │
└──────┬───────┘
│
┌──────────────────┼──────────────────┐
│ │ │
↓ ↓ ↓
```

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
| bus-mapping | | gadgets | | eth-types |
└──────┬───────┘  └──────┬───────┘  └──────┬───────┘
| | |
└──────────────────┼──────────────────┘
|
↓
┌──────────────┐
| halo2_proofs|
└──────────────┘


┌──────────────┐  ┌──────────────┐  ┌──────────────┐
| geth-utils | | zktrie | | mock |
└──────────────┘  └──────────────┘  └──────────────┘


┌──────────────┐  ┌──────────────┐  ┌──────────────┐
| aggregator | | prover | | testool |
└──────────────┘  └──────────────┘  └──────────────┘
```

의존성 계층:

1. **기반 계층**: `halo2_proofs`, `ethers-core`
2. **타입 계층**: `eth-types`, `gadgets`
3. **변환 계층**: `bus-mapping`, `zktrie`

4. **회로 계층**: `zkevm-circuits`

5. **증명 계층**: `prover`, `aggregator`

6. **테스트 계층**: `mock`, `testool`, `integration-tests`

7. **유틸리티 계층**: `geth-utils`, `external-tracer`

주요 외부 의존성

Halo2 생태계

- `halo2_proofs` : 영지식 증명 백엔드 (Scroll fork)
- `halo2curves` : 타원곡선 구현
- `halo2-base`, `halo2-ecc` : halo2-lib 라이브러리
- `halo2_gadgets` : 추가 Gadget

Scroll 특화

- `mpt-circuits` : MPT 회로 (외부 저장소)
- `misc-precompiled-circuit` : 프리컴파일 회로 (외부 저장소)
- `hash-circuit` : Poseidon 회로 (외부 저장소)

이더리움

- `ethers-core`, `ethers-providers`, `ethers-signers` : 이더리움 라이브러리 (Scroll fork)
- `revm-precompile`, `revm-primitives` : EVM 구현 (Scroll fork)

증명 검증

- `snark-verifier` : 증명 검증기 (Scroll fork)
- `snark-verifier-sdk` : SDK

Feature Flags

zkevm-circuits

- `scroll` : Scroll L2 기능 활성화
- `zktrie` : zkTrie 지원
- `poseidon-codehash` : Poseidon 코드해시
- `dual-bytecode` : 듀얼 바이트코드 테이블
- `parallel_syn` : 병렬 합성
- `test` : 테스트 기능
- `enable-stack/memory/storage` : 트레이스 추적

bus-mapping

- `scroll` : Scroll L2 기능
- `strict-ccc` : 엄격한 제약 조건 검사
- `retrace-tx` : 트레이스 재생성
- `enable-stack/memory/storage` : 트레이스 추적

빌드 프로파일

```
# Cargo.toml

[profile.bench]

opt-level = 3

debug = false

lto = "thin"
```

```
incremental = true


[profile.release]
opt-level = 3
debug = false
debug-assertions = true
overflow-checks = true
lto = "thin"
incremental = true
```

특징:

- Release 빌드에서도 debug-assertions 활성화
- Thin LTO로 빌드 시간과 성능 균형
- Incremental 컴파일 지원

관련 문서

이전/다음

- **이전**: 전체 구조 - 시스템 아키텍처 개요
- **다음**: 데이터 흐름 - 크레이트 간 데이터 흐름

같은 섹션

- 전체 구조 - 레이어별 시스템 구조
- 데이터 흐름 - EVM 실행부터 증명까지

관련 시스템

- Bus-Mapping 개요 - bus-mapping 크레이트 상세
- Witness 생성 - zkevm-circuits의 witness 모듈
- Halo2 통합 - Halo2 백엔드 사용

회로 구현

- EVM Circuit - zkevm-circuits의 핵심 회로
- State Circuit - 상태 관리 회로
- Gadget 시스템 - gadgets 크레이트

참고 자료

- `Cargo.toml` : Workspace 설정
- 각 크레이트의 `Cargo.toml` : 개별 크레이트 설정
- 각 크레이트의 `src/lib.rs` : 모듈 구조